Constraint Satisfaction Problem

```
def is_safe(n, board, row, col):
    for i in range(row):
        # Check same column or diagonals
        if board[i] == col or board[i] - i == col - row or board[i] + i == col + row:
            return False
    return True
def n_queen(n, board, row):
    if row == n:
        return [board[:]]
    else:
        solutions = []
        for col in range(n):
            if is_safe(n, board, row, col):
                board[row] = col
                solutions.extend(n_queen(n, board, row + 1))
        return solutions
def solve(n):
    board = [-1] * n
    return n_queen(n, board, 0)
# Example for n = 4
n = 4
solutions = solve(n)

# Print all solutions
print(f"Number of solutions for {n}-Queens: {len(solutions)}\n")

for index, solution in enumerate(solutions, start=1):
    for row in solution:
        print("".join(['Q' if row == col else '-' for col in range(n)]))
    print()
```

output :

```
Number of solutions for 4-Queens: 2
--Q-
Q---
---Q
-Q—

-Q--
---Q
Q---
--Q-
```

Game Search Algorithms

Minmax Algorithm :

```
import math
def minmax(curDepth, nodeIndex, maxTurn, Scores, targetDepth):
  if curDepth == targetDepth:
    return Scores[nodeIndex]
  if maxTurn:
    return max(
      minmax(curDepth + 1, nodeIndex * 2, False, Scores, targetDepth),
      minmax(curDepth + 1, nodeIndex * 2 + 1, False, Scores, targetDepth))
  else:
    return min(
      minmax(curDepth + 1, nodeIndex * 2, True, Scores, targetDepth),
      minmax(curDepth + 1, nodeIndex * 2 + 1, True, Scores, targetDepth))
scores = [3, 5, 6, 9, 1, 2, 0, -1]
treeDepth = int(math.log(len(scores), 2))
print("The optimal value is:", minmax(0, 0, True, scores, treeDepth))
output : The optimal value is: 5
```

Alpha-beta purning Algorithm :

```
import math
def alpha_beta_pruning(depth,nodeIndex,isMaximizingPlayer,values,alpha,beta,maxDepth):
 if depth == maxDepth:
  return values[nodeIndex]
 if isMaximizingPlayer:
  maxEval = -math.inf
  for i in range(3):
   eval = alpha_beta_pruning(depth+1,nodeIndex*3+i,False,values,alpha,beta,maxDepth)
   maxEval = max(maxEval,eval)
   alpha = max(alpha,eval)
   if beta <= alpha:
    break
  return maxEval
 else:
  minEval = math.inf
  for i in range(3):
   eval = alpha_beta_pruning(depth+1,nodeIndex*3+i,True,values,alpha,beta,maxDepth)
   minEval = min(minEval,eval)
   beta = min(beta,eval)
   if beta <= alpha:
    break
  return minEval
values = [3,5,6,9,1,2,0,7,4]
maxDepth = 2
optimal_value = alpha_beta_pruning(
  depth =0,
  nodeIndex = 0,
  isMaximizingPlayer = True,
  values = values,
  alpha = -math.inf,
  beta = math.inf,
  maxDepth = maxDepth
)
print("The optimal value is : ",optimal_value)
output : The optimal value is :  3
```

Burglary Alarm (Bayesian Belief Networks)

```
import random
from collections import defaultdict
# Bayesian Network representation
def make_burglary_alarm_bn():
    bn = {
        'Burglary': {
            'parents': [],
            'cpt': 0.001
        },
        'Earthquake': {
            'parents': [],
            'cpt': 0.002
        },
        'Alarm': {
            'parents': ['Burglary', 'Earthquake'],
            'cpt': {
                (True,  True): 0.95,
                (True,  False): 0.94,
                (False, True): 0.29,
                (False, False): 0.001
            }
        },
        'JohnCalls': {
            'parents': ['Alarm'],
            'cpt': {
                (True,): 0.90,
                (False,): 0.05
            }
        },
        'MaryCalls': {
            'parents': ['Alarm'],
            'cpt': {
                (True,): 0.70,
                (False,): 0.01
            }
        }
    }
    return bn
def prob_true(var, assignment, bn):
    node = bn[var]
    parents = node['parents']
    if not parents:  # root
        return node['cpt']
    parent_vals = tuple(assignment[p] for p in parents)
    return node['cpt'][parent_vals]
def enumeration_ask(X, evidence, bn):
    """
    Returns distribution over X given evidence as dict {True: p, False: p}
    """
    Q = {}
    for xval in [True, False]:
        extended = dict(evidence)
        extended[X] = xval
        Q[xval] = enumerate_all(list(bn.keys()), extended, bn)
    # normalize
    total = Q[True] + Q[False]
```

```
    return {True: Q[True] / total, False: Q[False] / total}

def enumerate_all(variables, evidence, bn):
    if not variables:
        return 1.0
    Y = variables[0]
    rest = variables[1:]
    if Y in evidence:
        p = prob_true(Y, evidence, bn) if evidence[Y] else 1 - prob_true(Y, evidence, bn)
        return p * enumerate_all(rest, evidence, bn)
    else:
        s = 0.0
        for y in [True, False]:
            evidence2 = dict(evidence)
            evidence2[Y] = y
            p = prob_true(Y, evidence2, bn) if y else 1-prob_true(Y, evidence2, bn)
            s += p * enumerate_all(rest, evidence2, bn)
        return s
def rejection_sampling(X, evidence, bn, N=10000):
    counts = {True: 0, False: 0}
    vars_order = list(bn.keys())
    for _ in range(N):
        sample = {}
        consistent=True
        for var in vars_order:
            p=prob_true(var,sample,bn)
            sample[var]=random.random()<p
            if var in evidence and sample[var]!=evidence[var]:
                consistent = False
                break
        if consistent:
            counts[sample[X]] += 1
    total=counts[True]+counts[False]
    if total==0:
        return {True:0.0,False:0.0}
    return {True:counts[True]/total,False:counts[False]/total}
def likelihood_weighting(X,evidence,bn,N=10000):
    weights = {True: 0.0, False: 0.0}
    vars_order = list(bn.keys())
    for _ in range(N):
        w = 1.0
        sample = {}
        for var in vars_order:
            if var in evidence:
                p=prob_true(var,sample,bn)
                if evidence[var]:
                    w *= p
                    sample[var]=True
                else:
                    w *= (1 - p)
                    sample[var]=False
            else:
                p = prob_true(var,sample,bn)
                sample[var] = random.random()<p
        weights[sample[X]] += w
    total = weights[True] + weights[False]
    if total == 0:
```

```
        return {True: 0.0, False: 0.0}
    return {True: weights[True]/total, False:weights[False]/total}
if __name__ == "__main__":
    random.seed(42)
    bn = make_burglary_alarm_bn()
    evidence = {'JohnCalls':True,'MaryCalls':True}
    print("Exact enumeration:")
    r_exact = enumeration_ask('Burglary', evidence, bn)
    print("  P(Burglary=True|J=True, M=True)={:.6f}".format(r_exact[True]))
    print("\nRejection sampling (N=100000):")
    r_rej = rejection_sampling('Burglary', evidence, bn, N=100000)
    print("  P(Burglary=True | J=True, M=True)≈{:.6f}".format(r_rej[True]))
    print("\nLikelihood weighting (N=10000):")
    r_lw = likelihood_weighting('Burglary', evidence, bn, N=20000)
    print(" P(Burglary=True|J=True,M=True)≈{:.6f}".format(r_lw[True]))
```

output : Exact enumeration:
  P(Burglary=True | J=True, M=True) = 0.284172

Rejection sampling (N=100000):
  P(Burglary=True | J=True, M=True) ≈ 0.221154

Likelihood weighting (N=10000):
  P(Burglary=True | J=True, M=True) ≈ 0.229139

HMM

```
import numpy as np
import matplotlib.pyplot as plt
from hmmlearn import hmm
import seaborn as sns
states=["Sunny","Rainy"]
n_states=len(states)
print('Number of hidden states:',n_states)
observations=["Dry","Wet"]
n_observations=len(observations)
print('Number of observations:',n_observations)
state_prob=np.array([0.6,0.4])
print("state probabilities:",state_prob)
trans_prob=np.array([[0.7,0.3],[0.3,0.7]])
print("transition probabilities:\n",trans_prob)
emission_prob=np.array([[0.9,0.1],[0.2,0.8]])
print("emission probabilities:\n",emission_prob)
model=hmm.CategoricalHMM(n_components=n_states)
model.startprob_=state_prob
model.transmat_=trans_prob
model.emissionprob_=emission_prob
observation_sequence=np.array([0,1,0,1,0,0]).reshape(-1,1)
observation_sequence
hidden_states=model.predict(observation_sequence)
print("hidden states:",hidden_states)
log_probability,hidden_states=model.decode(observation_sequence,lengths=len(observation_sequenc
e),algorithm="viterbi")
print("log probability:",log_probability)
print("hidden states:",hidden_states)
sns.set_style("whitegrid")
plt.plot(hidden_states,'-o',label="Hidden state")
```

```
plt.xlabel('Time step')
plt.ylabel('Most likely hidden state')
plt.title("Sunny or rainy")
plt.legend()
plt.show()
```

Thoery :

Constraint Satisfaction Problem :

N-queens:

You have a **chessboard of size N × N** (for example, 4×4 or 8×8).
You need to **place N queens** on the board so that **no two queens attack each other**.
Only one queen per row
→ Each row can have only one queen.
Only one queen per column
→ Each column can have only one queen.
No two queens on the same diagonal
→ Queens should not be placed in positions where they can attack diagonally (↘ or ↙).
Goal:
→ Place all N queens on the board following these rules.

Game search Minmax and Alpha-beta pruning

Feature	Minimax Algorithm	Alpha–Beta Pruning
Definition	A backtracking algorithm to find the optimal move for a player, assuming the opponent also plays optimally.	An optimization of Minimax that eliminates branches that don't affect the final decision.
Explores all nodes	Yes, checks **every possible move** in the game tree.	No, **skips (prunes)** unneeded branches.
Speed	Slower (checks all nodes).	Faster (fewer nodes checked).
Efficiency	Less efficient.	More efficient.
Result / Output	Gives the optimal move.	Gives the **same optimal move** as Minimax.
Uses pruning?	No.	Yes, uses α (alpha) and β (beta) to prune.
Extra parameters	None.	Alpha (maximum lower bound) and Beta (minimum upper bound).
Best case time complexity	O(b^d)	O(b^(d/2)) — nearly twice as fast in best case.
Where used	Two-player games (chess, tic-tac-toe, etc.)	Same — but preferred when tree is large.
Main goal	Find the optimal decision by exploring all moves.	Find the same optimal decision **more efficiently**.

Bayesian Belief Networks

Constructing Bayesian networks

A Bayesian networks can often be far more compact than the full joint probability distribution (provided that the network is sparse).

- assume that each random variable is directly influenced by at most k other variables (and we have n such variables); then the space of representation is
  - $n.2^k$</sup> for Bayesian network
  - 2^n for full joint distribution

We can also ignore some slight dependencies, which makes the network smaller in exchange for less accuracy

- for example, we assumed that call from Mary and John is driven by the alarm sound only but not for example by earthquake

naturally we will get a compact Bayesian network only if we choose the node ordering right

We have a **burglar alarm** installed at home. It is fairly reliable at detecting a **burglary**, but occasionally responds to minor **earthquakes**.
Our neighbors Mary and John promised to **call us** when they hear the alarm.

- John nearly always calls when he hears alarm, but sometimes confuses the telephone ringing with the alarm
- Mary likes loud music and often misses the alarm altogether

We would like to estimate the probability of a burglary **given the evidence of who has or has not called.**

HMM

Hidden Markov models

Assume that the state of process is described by a single discrete random variable X_t (there is also a single evidence variable E_t).
This is called a **hidden Markov model** (HMM).

This restricted model allows for a simple and elegant **matrix implementation** of all the basic algorithms.
Assume that variable X_t takes values from the set {1,...S}, where S is the number of possible states.
The **transition model** $\mathbf{P}(X_t \mid X_{t-1})$ becomes an S×S matrix **T**, where:

$$\mathbf{T}_{(i,j)} = P(X_t = j \mid X_{t-1}=i)$$ for the umbrella world $\begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}$

We also put the **sensor model** in matrix form. Now we know the value of the evidence variable e_t so we describe $P(E_t = e_t \mid X_t=i)$, using a diagonal matrix $\mathbf{O}_t$, where:

$$\mathbf{O}_{t\,(i,i)} = P(E_t = e_t \mid X_t=i)$$

$\mathbf{O}_1 = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.2 \end{pmatrix}$; $\mathbf{O}_3 = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.8 \end{pmatrix}$